

CodeAbsract: A Privacy Framework for Semantic Code Abstraction and Bidirectional Mapping in LLM Interaction

We are building a secure middleware layer between developers and AI coding models, allowing us to use LLMs without exposing raw source code. Instead of sending actual code, the system converts it into a logic-preserving abstract representation that keeps control flow, conditions, and error paths while hiding variable names, functions, secrets, and business logic. The LLM reasons on this abstracted logic and returns suggestions, which are then mapped back locally to the original code. This enables safe debugging, optimization, and code review with AI using a zero-trust approach, protecting intellectual property and security while still getting useful AI assistance.

Basic-Level

PROJECT: SecureVibe Privacy Middleware for AI Code Assistance

PROBLEM: Developers expose proprietary code to LLMs during "vibe coding," risking intellectual property theft and competitive disadvantage.

SOLUTION: A JWT-like abstraction layer that transforms sensitive code into semantically equivalent but non-revealing versions before sending to AI models, with bidirectional translation of responses.

INNOVATION: Unlike existing tools that only mask variable names (ObfusCat), we abstract the entire logic structure while preserving LLM utility through configurable privacy levels.

OUTCOME: Developers get AI assistance without exposing trade secrets, a practical tool for the AI-coding era.

SecureAbstract: Logic-Preserving Code Abstraction for Privacy-Safe LLM Interaction

Team: 4 Members | **Duration:** 6 Months | **Domain:** AI Security & Software Engineering

🎯 THE PROBLEM

Developers increasingly rely on AI coding assistants (ChatGPT, GitHub Copilot, Claude) for debugging and optimization. However, this requires **pasting proprietary source code** directly into cloud-based AI systems, exposing:

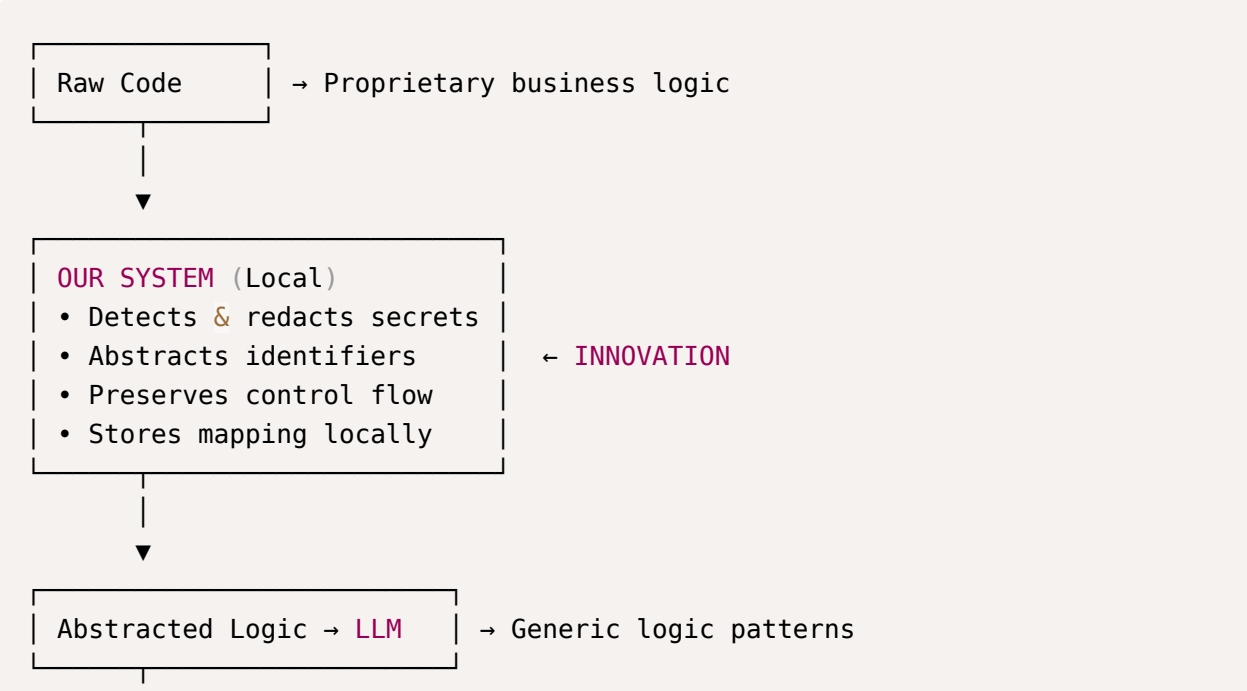
- 🔐 API keys and credentials
- 💰 Proprietary algorithms and business logic
- 🏢 Company intellectual property
- 📊 Strategic pricing/scoring formulas

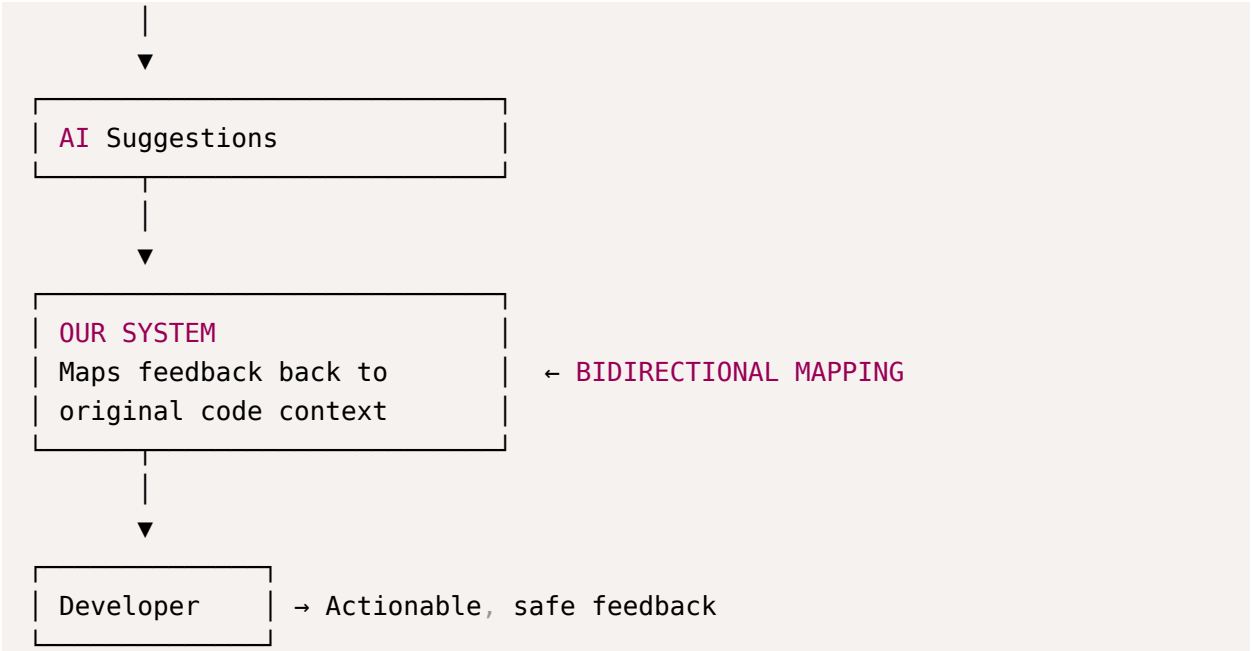
Current dilemma: Use AI and risk IP leakage **OR** avoid AI and lose productivity gains.

💡 OUR SOLUTION

A **middleware system** that converts source code into **logic-preserving abstractions** before sending to LLMs, enabling safe AI-assisted development without IP exposure.

How It Works:





Example Transformation:

Original Code	What AI Sees
`def calculate_stripe_commission(amount, tier):`	`FUNCTION: COMPUTE_VALUE(INPUT_A, CATEGORY_B)`
`if tier == "premium":`	`IF CATEGORY_B == CONSTANT_1:`
`return amount * 0.025`	`RETURN INPUT_A * 0.025`
Exposes: Stripe, commission structure	**Exposes:** Generic logic only

✨ NOVELTY & RESEARCH CONTRIBUTION

What Already Exists:

- **CodeCipher (2024):** Token obfuscation (`def` → `class`), requires LLM modification
- **Text anonymization:** PII removal from documents (not code logic)
- **LLM output security:** Securing AI-generated code (not input privacy)

Our Unique Contributions:

Aspect	Existing Work	Our Approach
Method	Token obfuscation	Semantic abstraction
Interpretability	Low (broken syntax)	High (structured logic)
Compatibility	Needs LLM access	Works with any API ✓
Mapping	One-way	**Bidirectional** ✓

Aspect	Existing Work	Our Approach
ML Enhancement	None	Sensitivity classifier ✓

Research Gap Filled: First semantic code abstraction framework with bidirectional mapping for privacy-preserving LLM interaction.

🛠️ TECHNICAL APPROACH

Core Components:

1. **AST Parser:** Analyzes code structure (Python `ast` library)
2. **Abstraction Engine:** Converts code → logic representation (rule-based + ML)
3. **Secret Detector:** Identifies API keys, credentials (fine-tuned CodeBERT)
4. **LLM Processor:** Sends abstracted code to OpenAI/Anthropic APIs
5. **Bidirectional Mapper:** Translates AI feedback back to original context
6. **ML Classifier (Optional):** Auto-detects code sensitivity (Random Forest)

Tech Stack:

- **Backend:** Python 3.10+, AST, NetworkX (CFG generation)
- **LLM APIs:** OpenAI GPT-4, Anthropic Claude
- **ML:** scikit-learn (Random Forest), transformers (CodeBERT)
- **Frontend:** Streamlit/Flask (web UI) or Click (CLI)

📊 EXPECTED OUTCOMES

Deliverables:

✅ **Working Prototype:** Python code abstraction tool (CLI + web interface)

✅ **Research Paper:** 10-12 pages targeting student conferences (ACM SIGCSE, IEEE CSEE)

✅ **Evaluation Study:** Privacy-utility trade-off analysis on 100+ code samples

✅ **Demo Video:** 5-minute demonstration with real-world examples

✅ **Open Source:** GitHub repository with documentation

Success Metrics:

- 🔐 **Privacy:** 85%+ identifier anonymization, 0 secrets leaked
- 🤖 **Utility:** LLM provides useful suggestions on 80%+ abstracted code
- 📈 **Performance:** <3 seconds to abstract 1000 lines of code
- 🎯 **Accuracy:** 85%+ mapping accuracy (feedback → original code)

⚖️ HONEST LIMITATIONS

What Our System Protects:

- ✅ Variable/function names, API keys, brand references, surface-level IP

What It Cannot Hide:

- ⚠️ Algorithm structure, mathematical formulas, business rule logic

Trade-off: We provide **configurable abstraction levels** (LOW/MEDIUM/HIGH) so users balance privacy vs utility.

📥📤 INPUT & OUTPUT EXAMPLE

INPUT (Developer's Original Code):

python

```
def process_stripe_payment(card_number, amount, api_key):
    """Charge customer via Stripe API"""

    stripe.api_key = "sk_live_abc123xyz"  # Secret!

    # Calculate 2.5% commission
    commission = amount * 0.025
    final_amount = amount + commission

    if final_amount > 10000:
        return stripe.charge(card_number, final_amount)
    else:
        raise ValueError("Amount too small")
```

Developer asks: *"Why does this fail for large amounts?"*

ABSTRACTED CODE (Sent to AI):

```
FUNCTION: PROCESS_TRANSACTION(IDENTIFIER_A, VALUE_B, SECRET_C)

  AUTH_TOKEN = [REDACTED]

  FEE = VALUE_B * 0.025
  TOTAL = VALUE_B + FEE
```

```
  IF TOTAL > 10000:
    RETURN EXTERNAL_API_CALL(IDENTIFIER_A, TOTAL)
  ELSE:
    THROW ERROR("Value too small")
```

Hidden from AI:
- ✅ Function name "stripe_payment"
- ✅ Variable names "card_number", "amount"
- ✅ API key "sk_live_abc123xyz"
- ✅ Business context "Stripe", "commission"

Still visible (needed for AI help):
- ⚠ The calculation logic (multiply by 0.025)
- ⚠ The threshold (10000)
- ⚠ The control flow (if/else)

AI RESPONSE (Based on Abstract Code):
```
ISSUE FOUND: Logic Error at Line 4

Problem: Your condition checks if TOTAL > 10000
But for large amounts (e.g., VALUE_B = 50000):
  TOTAL = 50000 + (50000 * 0.025) = 51250
  This PASSES the check when it should FAIL!

Your threshold logic is inverted.

Fix: Change to TOTAL < 10000 for the error case
```

OUTPUT (Mapped Back to Developer):
```
🔍 ANALYSIS RESULTS

═══════════════════════════════════

⚠ LOGIC ERROR FOUND

═══════════════════════════════════

Location: Line 9 (if final_amount > 10000)
```

```
Problem: Your threshold logic is inverted!
For large amounts like $50,000:
  final_amount = 50000 + 1250 = 51,250
  This PASSES when it should FAIL

Fix:
  if final_amount < 10000:  # Change > to
      raise ValueError("Amount too small")
  else:
      return stripe.charge(...)

🔒 Privacy: API key NOT exposed to AI
```

local table

- Research Paper Publication
- Empirically Evaluationof Algorithm
- 📘 Algorithm
- CLI IMPLEMENTATION
- Prompt to Build
- BIDIRECTIONAL MAPPING ALGORITHM
- Weekly Report
- Temp Report
- New Difference
- Example
- Credibility

- Methods
- Innovation
- Scenario
- Diagram
- Google spread Sheet (version)
- Reality Check
- Difference between
- Without-ML Part
- Hybrid Approach
- Working
- Research Papers
- End-End Approach
- Scenario
- Complete Idea(Difference between)
- On Company Perspective
- Google SpeardSheet

🧠 Key Idea

LLMs understand logic, not exact code.

So we send:

- Control flow
- Conditions
- Error paths

Not:

- Variable names
- Functions
- Secrets
- Business logic

That is a very timely and relevant concern. "Vibecoding"—moving fast and relying on AI for logic—often creates a "security-later" mindset that can lead to massive data leaks or intellectual property exposure.

Your idea of a **Secure AI Middleware** (a "Privacy Gateway") is a strong solution to a problem many enterprises are currently scrambling to fix.

The Core Problem: The "Context vs. Privacy" Paradox

LLMs require context to be helpful. If you encrypt the code entirely (like a standard AES encryption), the LLM just sees gibberish and can't help you debug. If you give it everything, you lose your secrets.

Your idea bridges this gap by creating a **structural representation** of the code rather than a literal one.

How the Secure Layer Would Work

Instead of a simple JWT-style encryption, you are looking at a process called **Pseudonymization** or **Context-Preserving Obfuscation**.

1. **Parsing (AST):** The layer breaks the code down into an Abstract Syntax Tree (AST). It identifies what "logic" is and what "identity " is."
2. **Tokenization:** It replaces sensitive identifiers (variable names, function names, company-specific strings) with generic tokens.
   - *Original:* `fetchUserBalance(apiKey: "prod_8822")`
   - *Obfuscated:* `func_A(var_B: "SECRET_VALUE")`
3. **LLM Processing:** The LLM receives the structural logic. It can see that `func_A` is trying to do a database call and can suggest improvements to the logic.
4. **De-masking:** When the LLM sends the code back, your layer maps `func_A` back to `fetchUserBalance` before the developer sees it.

Feature	Direct Paste (Vibecoding)	Your Secure Gateway
Secrets Exposure	High (API keys, DB URIs leaked)	Low (Automatically redacted)
IP Protection	None (Logic is stored in LLM history)	High (Variable names/intent are masked)
LLM Utility	100%	80-90% (Logic remains intact)
Compliance	Likely fails SOC2/GDPR	Built for compliance

At 1st researched

Feature	Existing Gateways (DLP)	Your Proposed Layer

Action	Blocks the prompt entirely.	**Obfuscates & Mappings back.**
Intelligence	Looks for "Keywords" (keys, IPs).	Looks for **"Structure"** (logic trees).
Developer Experience	Frustrating (Stop & Go).	**Seamless (Vibecoding friendly).**
Mapping Technique	Simple Redaction.	**JWT-inspired Tokenization.**

1. The Novelty: "Semantic-Preserved Structural Anonymization."

Standard obfuscators just rename `user_password` to `var_1`. A smart LLM can often guess that `var_1` is a password based on how it's used (e.g., if it's passed to a `login()` function).

Your Research Angle: Build a **Context-Aware AST (Abstract Syntax Tree) Transformer**.

- **The Innovation:** Instead of just renaming, your layer analyzes the code's *intent*. If the user is asking about "Performance Optimization," your layer replaces all business logic (the "what") with generic math operations but keeps the loops and data structures (the "how") intact.
- **Why it's Novel:** It creates a "Privacy-Utility Trade-off" model. You can write your paper on the mathematical correlation between "Percentage of Code Obfuscated" vs. "LLM Response Accuracy."

2. The "Privacy Score" (Adversarial Novelty)

A great research paper doesn't just build a tool; it **proves** the tool works.

- **The Innovation:** Integrate an "Attacker LLM" into your gateway. Once your layer obfuscates the code, the system sends it to a second LLM whose only job is to try and "De-anonymize" or guess the original business logic.
- **The Metric:** You introduce a new metric, let's call it the **Code Privacy Entropy (CPE)**. Your research paper would focus on how your layer maximizes CPE while minimizing the drop in the LLM's helpfulness.

3. "Differential Privacy" for Code Snippets

Differential privacy is usually for databases, but applying it to **code structure** is very modern.

- **The Innovation:** Inject "Structural Noise." For every 5 lines of real (but obfuscated) code, your layer could inject 2 lines of "junk logic" that looks real but means nothing. This prevents the LLM from "memorizing" the exact proprietary algorithm of your company.
- **The Research Gap:** Most researchers focus on text, while very few apply formal differential privacy to the **syntax trees** of programming languages.

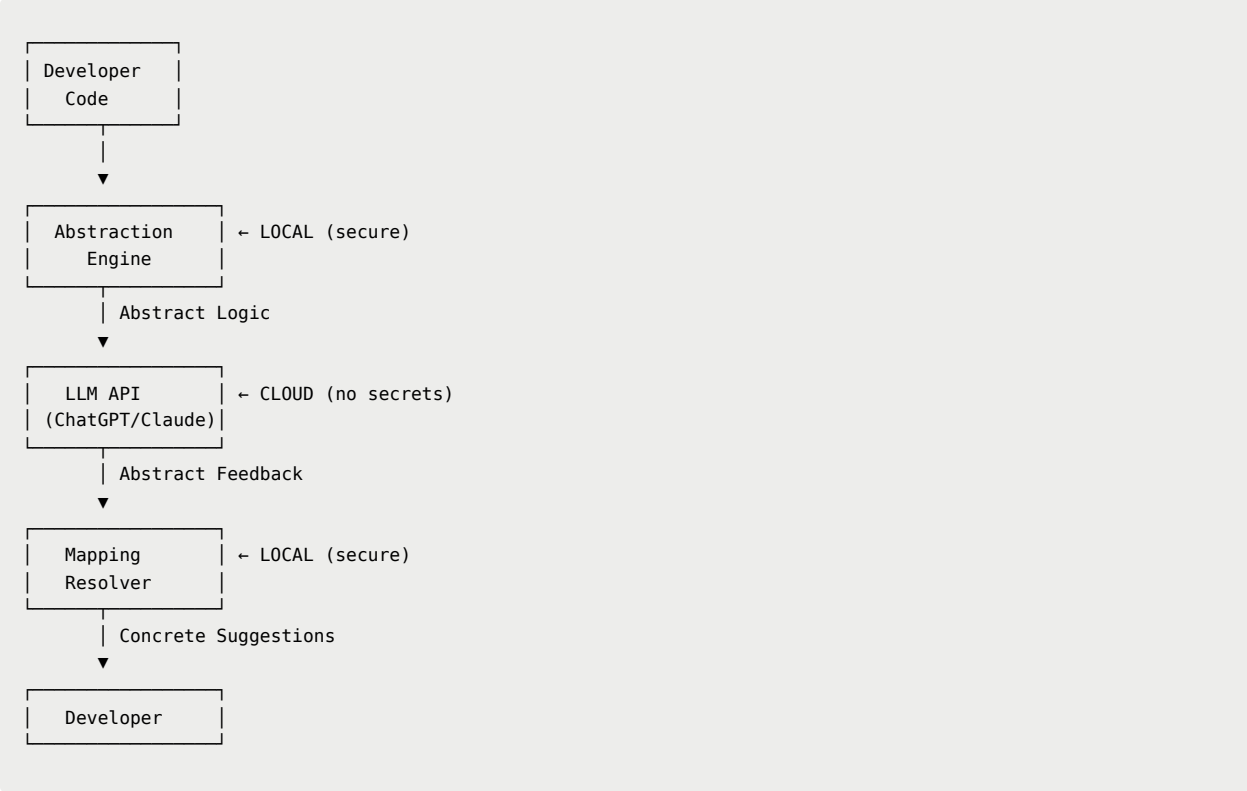
At 2nd Researched

Feature	Basic Gateway (Not Novel)	Your Project (Research Grade)
Method	RegEx Redaction (Find & Replace)	**AST-based Tokenization**
Mapping	Static JSON file	**Dynamic State-Store (JWT-inspired)**

Logic Hiding	None	**Semantic Logic Replacement**
Validation	Manual testing	**Adversarial Re-identification Attack**

Goal: Send abstract code to LLM, get abstract feedback

Architecture:



The Input/Output Lifecycle

Stage	Input	Action (Your Logic)	Output
1. The Uplink (Privacy Guard)	**Raw Private Code:** Contains actual function names, API keys, and business logic.	AST Parsing + Secret Detection + Variable Mapping.	**Obfuscated Code:** Generic names (`func_01` , `var_A`) sent to the LLM.
2. The LLM Processing	**Obfuscated Code:** The AI sees only the structural logic.	The AI processes the request (e.g., "Fix this bug" or "Optimize").	**AI Suggestion (Obfuscated):** The fix, still using the generic tokens.
3. The Downlink (Reconstruction)	**AI Suggestion (Obfuscated):** Code that works but uses "func_01" names.	Inverse Mapping using your local "Vault" (the JWT-like map).	**Final Clean Code:** The fix, but with your original names restored.

3-Stage Pipeline:

```
┌─────────────────────────────────────────────────────┐
│ STAGE 1: PRIVACY GUARD (Your Innovation)            │
├─────────────────────────────────────────────────────┤
│ Input:  Raw source code with secrets                │
│ Action: 1. AST Parsing                              │
│         2. Secret Detection (API keys, passwords)   │
│         3. Variable Mapping (local JWT-like vault)  │
│         4. Logic Abstraction (control flow extraction)  │
│ Output: Abstract logic representation               │
└─────────────────────────────────────────────────────┘
                          ↓
┌─────────────────────────────────────────────────────┐
│ STAGE 2: LLM PROCESSING (Standard)                  │
├─────────────────────────────────────────────────────┤
│ Input:  Abstracted logic                            │
│ Action: LLM analyzes logic, suggests improvements   │
│ Output: AI suggestions in abstract terms            │
└─────────────────────────────────────────────────────┘
                          ↓
┌─────────────────────────────────────────────────────┐
│ STAGE 3: RECONSTRUCTION (Your Innovation)           │
├─────────────────────────────────────────────────────┤
│ Input:  AI suggestions + Local mapping vault        │
│ Action: 1. Parse LLM response                       │
│         2. Inverse mapping using vault              │
│         3. Contextualize suggestions                │
│         4. Apply to original code                   │
│ Output: Clean, actionable feedback                  │
└─────────────────────────────────────────────────────┘
```

FINAL USER OUTPUT (What developer sees)

CLI Output

```
$ python secure-llm-tool.py debug payment.py --function process_payment

🔒 Abstracting code... ✓
🤖 Querying LLM... ✓
🔄 Mapping feedback... ✓

═══════════════════════════════════════════════════
                    ANALYSIS COMPLETE
═══════════════════════════════════════════════════

Found 3 potential issues:

1. ⚠  Race Condition (Line 8-19)
2. 🔴 Floating Point Bug (Line 7) [CRITICAL]
```

```
3. ⚠  Missing Validation (Before line 7)

[Full details above]

💡 Apply suggested fixes? [Y/n]: _
```

Option B: Web UI Output
```
┌─────────────────────────────────────────────────┐
│  Secure Code Analysis                           │
│                                                 │
│  ✓ Code abstracted (0 secrets exposed)          │
│  ✓ LLM analysis complete                        │
│  ✓ Feedback mapped to original code             │
└─────────────────────────────────────────────────┘

📊 Security Report:
   ✓ No variable names sent
   ✓ No business logic exposed
   ✓ No identifiers leaked

🐛 Issues Found: 3

[Interactive UI with code highlighting and suggestions]
```

Component	INPUT	OUTPUT
Module A: Pre-processor	Raw Source Code + User Query	Cleaned Code (Secrets removed)
Module B: AST Abstractor	Cleaned Code	**1.** Abstracted Logic Skeleton **2.** Local Mapping Table
Module C: LLM Interface	Abstracted Logic Skeleton + Query	AI Response (using abstract tokens)
Module D: Re-mapper	AI Response + Local Mapping Table	**Final Actionable Feedback** (with original names)

INPUT FORMATS:

1. **Source Code** (required)
2. **User Query** (optional - defaults to "analyze this code")
3. **Sensitivity Level** (optional - AUTO/LOW/MEDIUM/HIGH/CRITICAL)
4. **File Metadata** (filename, language, etc.)

OUTPUT FORMATS:

1. **Abstracted Code** (sent to LLM)
2. **Local Vault** (NEVER sent, stays on device)
3. **LLM Analysis** (raw AI response)
4. **Mapped Feedback** (translated back to original code)
5. **Privacy Report** (what was hidden/exposed)

📊 USE CASE SUMMARY TABLE

Use Case	User	Problem	Solution	Value
Debugging	Developer	Can't share production code with AI	Auto-redact secrets	Safe AI assistance
Learning	Student	Accidentally posts secrets online	Clean before sharing	Prevent breaches
Collaboration	Team lead	External contractors need code access	Abstract sensitive parts	Secure outsourcing
AI Tools	Developer	GitHub Copilot sees secrets	Intercept & redact	Safe AI usage
Compliance	Security team	Prove no data exposure	Audit logs	Regulatory compliance
Education	Professor	Students expose university keys	Batch sanitization	Institutional protection
Open Source	Maintainer	Secrets in codebase history	Scan & clean	Safe publishing

Some Random Stuff with Extra

PROBLEM STATEMENT

Developers using AI coding assistants (ChatGPT, GitHub Copilot, Claude) frequently paste code containing:

- API keys and authentication tokens
- Database credentials
- Proprietary business logic
- Customer data

This creates security vulnerabilities and compliance risks.

PROPOSED SOLUTION

A local-first privacy layer that:

1. Detects sensitive data in code (regex + pattern matching)
2. Replaces secrets with safe placeholders
3. Sends anonymized code to AI services
4. Maps AI responses back to original context

```
EXAMPLE WORKFLOW:

┌──────────────────────────────────────────────┐
│ Developer Code:                              │
│ stripe.api_key = "sk_live_abc123"            │
│ charge = stripe.Charge.create(amount=1000)   │
└──────────────────────────────────────────────┘

↓

┌──────────────────────────────────────────────┐
│ Sent to AI:                                  │
│ service.api_key = "[REDACTED_KEY_1]"         │
│ result = service.API_CALL(amount=1000)       │
└──────────────────────────────────────────────┘

↓

┌──────────────────────────────────────────────┐
│ AI Response (anonymized):                    │
│ "Add error handling around API_CALL"         │
└──────────────────────────────────────────────┘

↓

┌──────────────────────────────────────────────┐
│ Developer Sees:                              │
│ "Add error handling around stripe.Charge"    │
└──────────────────────────────────────────────┘
```

TECHNICAL APPROACH

Core Technologies:

- Python (AST parsing, regex)
- Anthropic/OpenAI APIs
- AES-256 encryption (local vault)

System Components:

1. Secret Detector - Pattern matching for API keys, passwords

2. Code Abstractor - Variable/function anonymization
3. Local Vault - Encrypted mapping storage
4. API Interface - Communication with LLM services
5. Response Mapper - Bidirectional translation

TIMELINE (24 Weeks)

Weeks 1-8: Core Engine Development
- Secret detection patterns
- Abstraction logic
- Local vault encryption

Weeks 9-16: Integration & Interface
- CLI tool
- LLM API integration
- Response mapping

Weeks 17-20: Evaluation
- User testing (10 developers)
- Metrics collection
- Performance analysis

Weeks 21-24: Documentation & Defense
- Thesis writing
- Final presentation prep

DELIVERABLES

Software:
✓ Command-line tool (Python package)
✓ Open-source repository (GitHub)
✓ Documentation and usage guide

Academic:
✓ Thesis document (40-60 pages)
✓ Evaluation report with metrics
✓ Defense presentation

Evaluation Metrics:
✓ Secret detection accuracy (target: 95%+)
✓ AI utility preservation (target: 80%+)
✓ User satisfaction (target: 7/10+)

NOVELTY & CONTRIBUTION

Compared to existing solution (ObfusCat):

- Open source vs. proprietary
- Research evaluation vs. commercial claims
- Free vs. paid
- Academic rigor vs. marketing

Academic Contribution:

- Empirical evaluation of abstraction strategies
- Privacy-utility trade-off measurement
- Open dataset for future research

RISK MITIGATION

Risk: Abstraction reduces AI helpfulness
Mitigation: Multiple abstraction levels (LOW/MEDIUM/HIGH)
User can choose privacy vs. utility trade-off

Risk: False positives (detecting non-secrets as secrets)
Mitigation: Whitelist system + user override option

Risk: Running behind schedule
Mitigation: Phased approach - Phase 1 is MVP,
Phases 2-3 are enhancements

SUCCESS CRITERIA

Must Achieve:
✓ Working CLI tool that processes Python code
✓ Detects common secrets (API keys, passwords, tokens)
✓ Integrates with at least one LLM (ChatGPT or Claude)
✓ Bidirectional mapping functional
✓ Complete thesis document

Stretch Goals:
⭐ Web interface
⭐ Support for JavaScript/TypeScript
⭐ IDE plugin (VS Code)
